



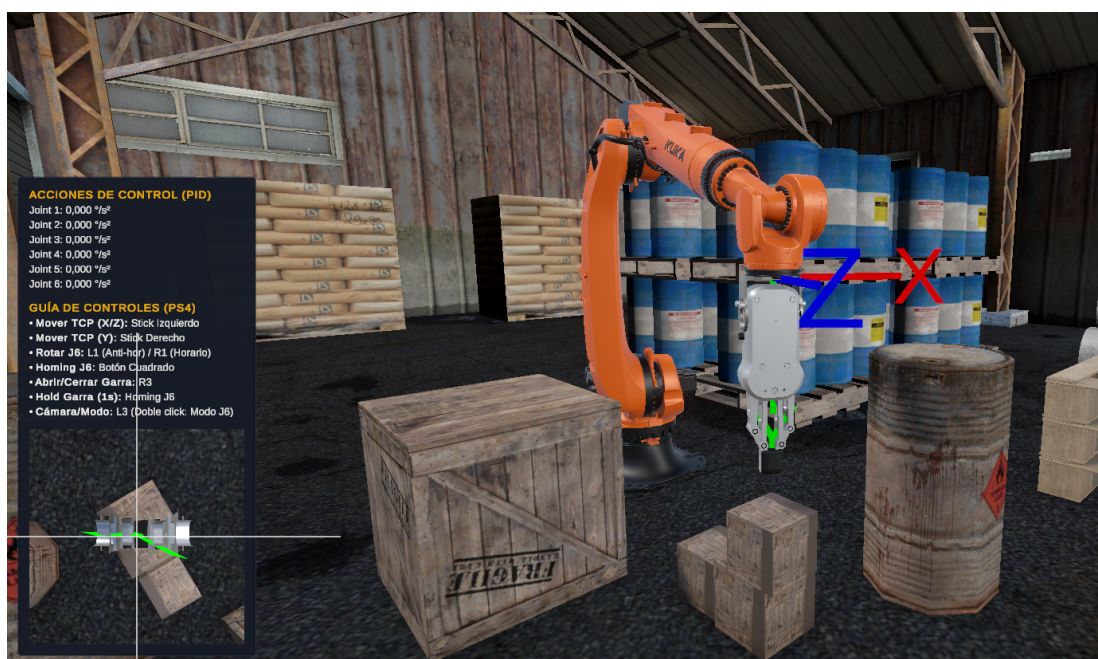
UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



**FACULTAD
DE INGENIERÍA**

Entrenamiento en entorno virtual para la teleoperación segura de Brazos Robóticos

Realidad Virtual – Facultad de Ingeniería



Profesores

Ing. Javier Rosenstein

Ing. Carlos Tomba

Alumnos

Quiroga, Juan Ignacio – 13889

Castel, Francisco – 13784

Índice

1	Introducción y Motivación	2
1.1	Descripción del problema	2
1.2	Motivación para el entorno de Realidad Virtual	2
2	Objetivos y Alcance	3
3	Hardware de interfaz: Joystick RPi Pico	4
4	Desarrollo en Unity	5
4.1	Listado de Comandos y Canvas	6
4.2	Remapeo Geométrico de Direcciones	10
4.3	Control PID en Articulaciones	11
4.4	Movimiento del Gripper y Realimentación Háptica	22
4.5	Comunicación UDP con Arquitectura Servidor–Cliente	34

1 Introducción y Motivación

1.1 Descripción del problema

La empresa **AUTOELEC** se dedica al asesoramiento, venta, reparación y alquiler de autoelevadores y baterías. Dentro de su proceso productivo, una tarea crítica es el ensamble de paquetes de baterías de plomo-ácido destinadas a autoelevadores industriales. Dichos paquetes se construyen ubicando manualmente múltiples vasos (celdas individuales) de entre 20 y 100 kg dentro de un rack metálico, conformando la batería final que alimenta el vehículo.



Figura 1: Vastos de plomo-ácido dispuestos en rack previo al ensamble.



Figura 2: Paquete de baterías terminado, marca Eternity – distribuido por AUTOELEC.

Al tratarse de un proceso completamente manual, el operador debe elevar cada vaso asistido únicamente por su propia fuerza física y ubicarlo en una posición determinada dentro del rack. A medida que el rack se va completando, el espacio libre se reduce notablemente, lo que obliga al operador a trabajar en posiciones forzadas durante la inserción de las últimas unidades. Esta combinación de esfuerzo físico considerable, movimientos repetitivos y posturas comprometidas genera riesgos ergonómicos que provocan un desgaste prematuro en la salud del personal, además de representar un riesgo concreto de accidentes durante la operación.

1.2 Motivación para el entorno de Realidad Virtual

La teleoperación y el aprendizaje de sistemas de manipulación robótica presentan desafíos particulares en contextos industriales. Durante la fase de entrenamiento existe riesgo de colisiones con

el rack, caída de vasos y golpes al operador; al mismo tiempo, la tarea exige una precisión manual considerable en el posicionamiento de piezas pesadas, y la destreza inicial varía significativamente entre distintos operadores.

Un entorno de **Realidad Virtual (VR)** permite abordar estos desafíos de forma integral. En primer lugar, el operador aprende a manejar el manipulador sin riesgo de dañar equipos ni lastimarse, al interactuar con una réplica virtual del sistema real. Esto elimina a su vez los costos asociados a errores durante el entrenamiento, como materiales dañados o paradas de producción no planificadas. Además, el entorno virtual puede reconfigurarse libremente para distintas secuencias de ensamble, cargas o restricciones sin ninguna intervención física. Desde el punto de vista de la evaluación del desempeño, la plataforma permite instrumentar automáticamente indicadores objetivos como el tiempo por tarea, la suavidad del movimiento, la cantidad de colisiones y los reintentos necesarios. Finalmente, el joystick físico utilizado durante el entrenamiento virtual es el mismo con el que se opera el robot real, lo que facilita la transferencia directa de los patrones de movimiento aprendidos en VR al control del equipo físico.

En este contexto, el presente proyecto propone el desarrollo de un entorno VR para entrenar operarios en la teleoperación de un brazo robótico industrial, utilizando un joystick diseñado ad hoc sobre *Raspberry Pi Pico* como interfaz de control y el motor gráfico *Unity* como plataforma de simulación e interacción.

2 Objetivos y Alcance

El objetivo general del proyecto es desarrollar un entorno VR y un esquema de control que permitan entrenar y luego teleoperar un brazo robótico mediante un joystick propio, priorizando seguridad, usabilidad y desempeño. Para lograrlo, el trabajo se organizó en torno a un conjunto de objetivos específicos que abarcaron tanto el software como el hardware del sistema.

En el plano del software, se buscó implementar un simulador del brazo robótico en Unity con interacción XR, diseñar el mapeo de los controles del joystick a los grados de libertad del robot, e incorporar mecanismos de seguridad como parada de emergencia, zonas prohibidas y límites de velocidad. Complementariamente, se desarrollaron herramientas de monitoreo externo mediante comunicación UDP, que permiten visualizar el estado articular del robot desde distintos dispositivos en tiempo real. En el plano del hardware, el objetivo fue construir un prototipo funcional del joystick RPi Pico que opera como dispositivo HID estándar, incluyendo la capacidad de emitir retroalimentación háptica hacia el operador.

El alcance del proyecto comprende el entrenamiento VR de tareas representativas de la aplicación industrial —en particular, el posicionamiento preciso del efector final sobre las celdas de batería—, la teleoperación con el joystick RPi Pico mediante comunicación USB, la integración de la capa de comunicación UDP para clientes externos y la implementación de una capa básica de seguridad en la simulación. Quedan fuera del alcance de esta versión la retroalimentación háptica proporcional a la fuerza de contacto, el *motion planning* autónomo, la integración física directa con el robot electrohidráulico real —que se reserva como trabajo futuro— y cualquier certificación normativa

industrial.

3 Hardware de interfaz: Joystick RPi Pico

Se diseñó y construyó un controlador de dos ejes analógicos con carcasa ergonómica impresa en 3D, cuyo núcleo es un microcontrolador **Raspberry Pi Pico**. El diseño aprovecha los conversores analógico-digitales (ADC) integrados en el microcontrolador para la lectura de los potenciómetros de cada eje, y su interfaz USB nativa para la comunicación con el host. La carcasa fue modelada e impresa en PLA, adoptando una forma bimanual que permite sostener el dispositivo de forma natural durante sesiones prolongadas.

El firmware fue desarrollado en C/C++ sobre el SDK oficial de RPi Pico. Para la capa de comunicación se implementó el protocolo **USB HID** utilizando la librería *TinyUSB*, lo que permite que el joystick sea reconocido automáticamente por el sistema operativo como un gamepad estándar sin necesidad de drivers adicionales. Esta decisión simplifica significativamente la integración con Unity: el *Input System* del motor detecta el dispositivo y expone sus ejes y botones directamente en el editor, sin código de inicialización adicional. El funcionamiento correcto fue verificado con la herramienta *Gamepad Tester*, que confirmó la lectura en tiempo real de los ejes analógicos y la identificación del dispositivo como *cafe-4004-Joystick VR*.

Además de la lectura de ejes, el firmware implementa la capacidad de **recepción de comandos de vibración** provenientes de Unity. A través de la misma interfaz USB HID, la aplicación puede enviar un comando de activación del motor háptico incorporado en el joystick, el cual se utiliza para alertar al operador sobre la proximidad del efector final a un objeto antes del contacto físico.

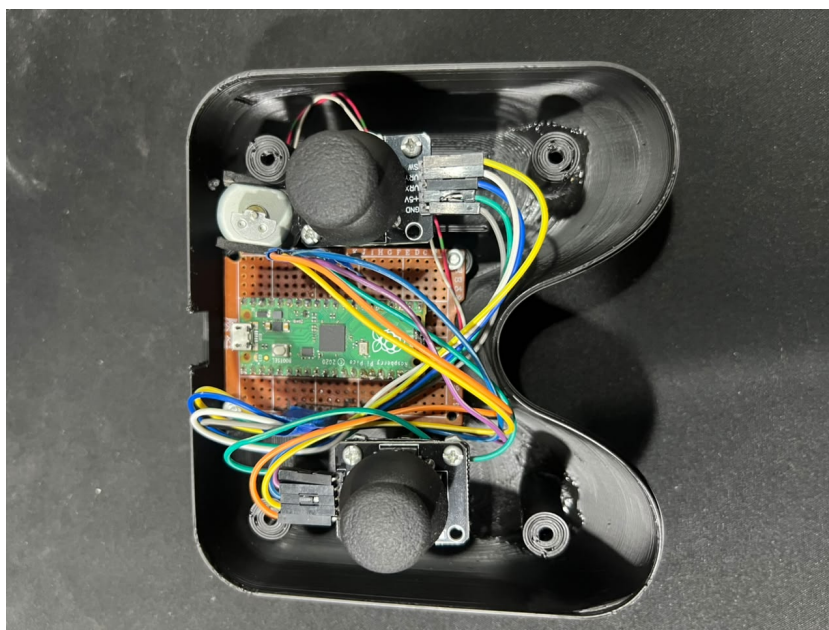


Figura 3: Interior del joystick: RPi Pico montado sobre placa de prototipado con los módulos analógicos de cada eje.



Figura 4: Vista exterior del prototipo con carcasa impresa en 3D y thumbsticks montados.

El estado actual del prototipo es funcional para todas las operaciones requeridas por la aplicación: lectura de los dos ejes analógicos, detección de botones y retroalimentación háptica. Las iteraciones futuras contemplan el refinamiento de la zona muerta (*deadband*), la aplicación de filtrado de ruido en los ADC y mejoras ergonómicas adicionales sobre la carcasa.

4 Desarrollo en Unity

La aplicación de entrenamiento fue desarrollada en **Unity 6000.3 LTS** utilizando el paquete de robótica **Flange** (`com.prelly.flange`) para la simulación cinemática del brazo robótico. Flange provee el modelo 3D articulado del **KUKA KR210 R3100-2**, un solver de cinemática inversa analítica integrado y una interfaz de control accesible desde C#, lo que evita la necesidad de implementar el cálculo cinemático desde cero. El joystick se integra al motor a través del *Unity Input System*, que reconoce el dispositivo como un gamepad HID estándar y expone sus ejes y botones mediante un *Input Action Asset* configurable desde el editor.

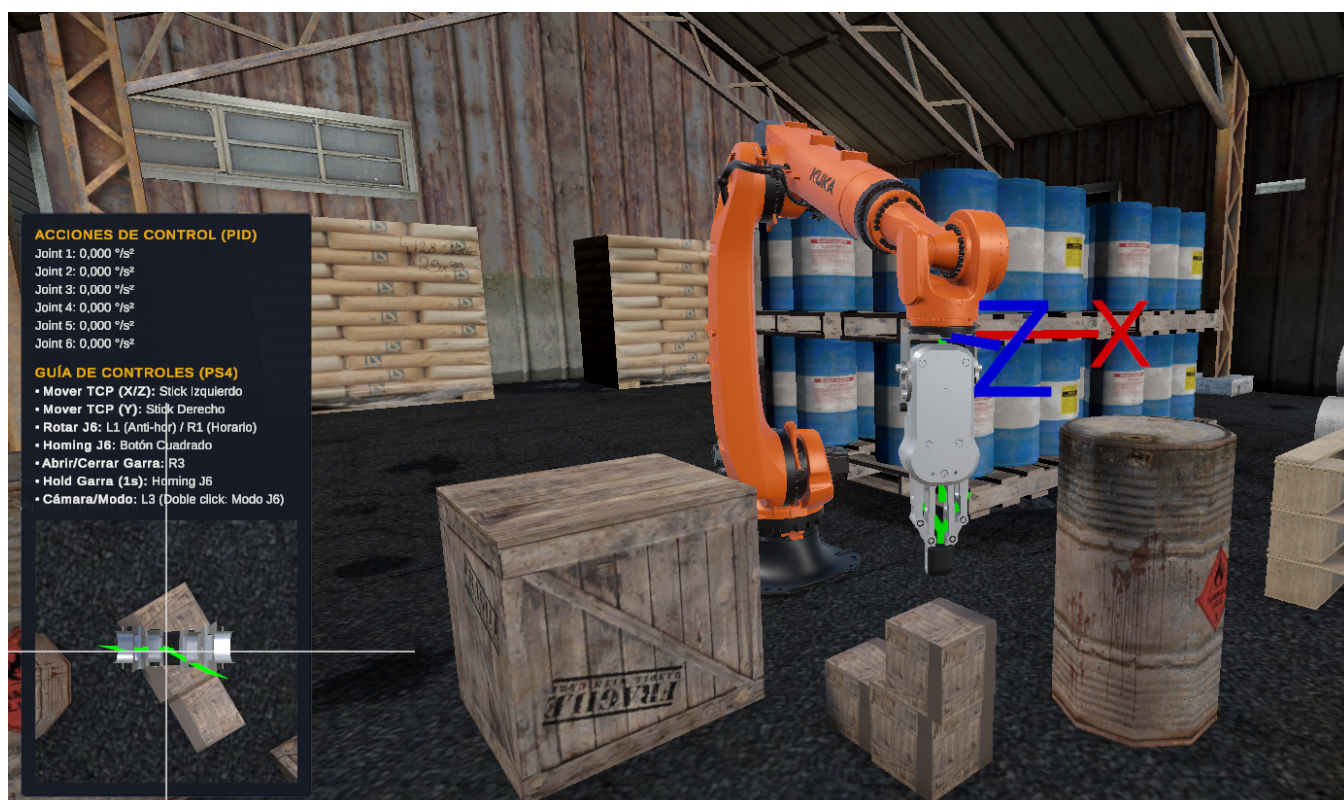


Figura 5: Escena de simulación en Unity: brazo KUKA KR210 con efector final tipo gancho integrado mediante el paquete *Flange*. Al fondo se aprecia el rack de ensamble y otros elementos del entorno industrial virtual.

4.1 Listado de Comandos y Canvas

La interfaz de usuario en pantalla (*Canvas*) fue implementada utilizando el sistema UI de Unity y permanece visible durante toda la sesión de entrenamiento. El panel se organiza en dos bloques principales, visibles en la Figura 6: el bloque de **Acciones de Control (PID)**, que muestra en tiempo real los valores de acción de control ($^{\circ}/s^2$) calculados por el controlador PID de cada articulación (J1–J6), y la **Guía de Controles (PS4)**, que lista el mapeo de botones y palancas del joystick para referencia inmediata del operador. En la esquina inferior izquierda se presenta la **cámara del gripper**, una sub-ventana que muestra la vista en tiempo real desde una cámara montada sobre el efector final, facilitando el posicionamiento preciso de la garra sobre las celdas de batería.



Figura 6: Panel de interfaz (*Canvas*) en ejecución: bloque de **Acciones de Control (PID)** con las acciones calculadas por articulación (J1–J6), guía de comandos del joystick PS4 y vista en tiempo real de la cámara embebida en el gripper (esquina inferior izquierda).

Las acciones de entrada se definen en el *Input Action Asset* `PS4Joystick.inputactions`. Las Tablas 1 y 2 resumen el mapeo físico completo: la primera corresponde al perfil del joystick HID propio (VR-2) y la segunda al perfil alternativo para gamepad PlayStation 4. El sistema opera en dos modos exclusivos, conmutados mediante la acción `ModoCamara` (L3): en **modo Robot** (`IsCameraMode = false`), los analógicos mueven el *Tool Center Point* (TCP) del manipulador en el espacio cartesiano; en **modo Cámara** (`IsCameraMode = true`), los mismos analógicos controlan la posición y orientación de la cámara principal al estilo *first-person*. Al tratarse de una propiedad estática en `JoystickAdapter`, el componente `CameraJoystickController` —adjunto a la *Main Camera*— puede consultarla sin requerir una referencia explícita entre objetos.

Tabla 1: Mapeo completo del joystick HID propio VR-2 (RobotBasic.inputactions).

Acción	Tipo	Control VR-2 / HID	Uso en la aplicación
Mover X	Value	Palanca principal, eje <code>stick/y</code>	Traslación lateral del TCP; en modo Cámara, avance/retroceso.
Mover Y	Value	Palanca de altura, eje <code>z</code>	Traslación vertical del TCP.
Mover Z	Value	Palanca principal, eje <code>stick/x</code>	Traslación frontal del TCP; en modo Cámara, desplazamiento lateral. Junto con Mover X forma el gesto circular para J6.
Gripper	Button	Trigger izquierdo (<code>trigger</code>)	Pulsación corta: alterna apertura/cierre al soltar. Mantener 1s: referencia J6 sin accionar la garra.
Camera Toggle	Button	Pulsador derecho (<code>button2</code>)	Clic corto: Robot/Cámara; doble clic: modo J6; mantener 2s: abre o cierra el menú de pausa.
Click	Button	Pulsador derecho (<code>button2</code>)	Confirma la opción seleccionada cuando el menú está abierto.
SelectorUP/DOWN	Value	Palanca de altura (<code>z</code>)	Navegación vertical del menú.
SelectorLEFT/RIGHT	Value	Palanca principal (<code>stick/y</code>)	Navegación horizontal del menú.
Pausa	Button	Trigger izquierdo (<code>trigger</code>)	Acción reservada en el asset; la pausa efectiva del perfil VR-2 se ejecuta manteniendo Camera Toggle .

Tabla 2: Mapeo completo del perfil alternativo PlayStation 4 (`PS4Joystick.inputactions`).

Acción	Tipo	Control PS4	Uso en la aplicación
MoveX / MoveZ	1D Axis	Stick izquierdo	Traslación lateral/frontal del TCP; también forma el gesto circular de J6.
MoveY	1D Axis	Stick derecho, eje vertical	Traslación vertical del TCP.
MoveForward / MoveSide	1D Axis	Stick izquierdo	Avance/retroceso y desplazamiento lateral de la cámara.
ViewUp / ViewSide	1D Axis	Stick derecho	Rotación vertical (<i>pitch</i>) y horizontal (<i>yaw</i>) de la cámara.
ModoCamara	Button	L3	Clic corto: Robot/Cámara; doble clic: modo J6; mantener 2 s: pausa/menú.
Gripper	Button	R3	Pulsación corta: apertura/cierre; mantener 1 s: referencia J6.
J6AntiHor / J6Hor	Button	L1 / R1	En modo J6, giro antihorario/horario a velocidad constante.
J6Home	Button	Cuadrado	Retorno directo de J6 a su referencia física.
Click	Button	Cruz	Confirmación de la opción seleccionada.
Pause	Button	Options	Abre o cierra el menú de pausa.
SelectorUP / SelectorDOWN	Button	Cruceta arriba/abajo	Navegación vertical del menú.
SelectorLEFT / SelectorRIGHT	Button	Cruceta izquierda/derecha	Navegación horizontal del menú.

En modo Robot, el componente `JoystickAdapter` construye en cada `FixedUpdate` el incremento de posición del TCP en espacio mundo:

$$\Delta \mathbf{p} = (\hat{\mathbf{d}}_X \cdot u_X + \hat{\mathbf{Y}}_{\text{mundo}} \cdot u_Y + \hat{\mathbf{d}}_Z \cdot u_Z) \cdot v \cdot \lambda_{\text{payload}} \cdot \Delta t \quad (1)$$

donde $u_X, u_Y, u_Z \in [-1, 1]$ son los valores leídos de las acciones `MoveX`, `MoveY` y `MoveZ`; v es la velocidad máxima configurada en el Inspector; $\hat{\mathbf{d}}_X$ y $\hat{\mathbf{d}}_Z$ son los versores mundo asignados por el remapeo geométrico (Sección 4.2) con signo fijo +1; y $\lambda_{\text{payload}} \in (0, 1]$ es un factor de penalización proporcional a la masa del objeto agarrado —implementado en `JoystickAdapter` como `payloadSpeedMultiplier`— que reduce la velocidad cartesiana de la consigna conforme aumenta la carga, produciendo el efecto perceptible de “carga pesada = movimiento más lento” en VR. El incremento se aplica sobre la pose actual del TCP y se pasa al solver de cinemática inversa de Flange; si la solución es válida, se actualizan las consignas articulares.

4.2 Remapeo Geométrico de Direcciones

Cuando el operador navega libremente por la escena en modo Cámara y luego retoma el control del robot, la dirección “al frente” del punto de vista puede no coincidir con ningún eje de movimiento predefinido. Para que los analógicos siempre empujen el TCP en la dirección intuitiva —aquella que apunta directamente del operador hacia el efector— se implementó el método `RemapAxesFromCamera()` en `JoystickAdapter`.

El algoritmo calcula primero el vector $\vec{C}$ desde la posición de la cámara hasta la posición del TCP, anula su componente vertical y lo normaliza para obtener una dirección puramente horizontal en el plano XZ del mundo:

$$\hat{\mathbf{d}}_Z = \frac{(\mathbf{p}_{\text{TCP}} - \mathbf{p}_{\text{cam}})|_{Y=0}}{\|(\mathbf{p}_{\text{TCP}} - \mathbf{p}_{\text{cam}})|_{Y=0}\|} \quad (2)$$

Este versor se asigna directamente a la dirección de avance de la palanca `MoveZ` con signo positivo fijo. La dirección lateral de `MoveX` se obtiene como el producto vectorial entre el eje $+Y$ del mundo y $\hat{\mathbf{d}}_Z$:

$$\hat{\mathbf{d}}_X = \frac{\hat{\mathbf{Y}}_{\text{mundo}} \times \hat{\mathbf{d}}_Z}{\|\hat{\mathbf{Y}}_{\text{mundo}} \times \hat{\mathbf{d}}_Z\|} \quad (3)$$

también con signo positivo fijo. De este modo, empujar el analógico izquierdo hacia adelante siempre mueve el TCP en la dirección que apunta hacia él desde la cámara, y empujarlo a la derecha lo desplaza a su derecha geométrica, con independencia de la orientación del efector o de la posición de la cámara en la escena.

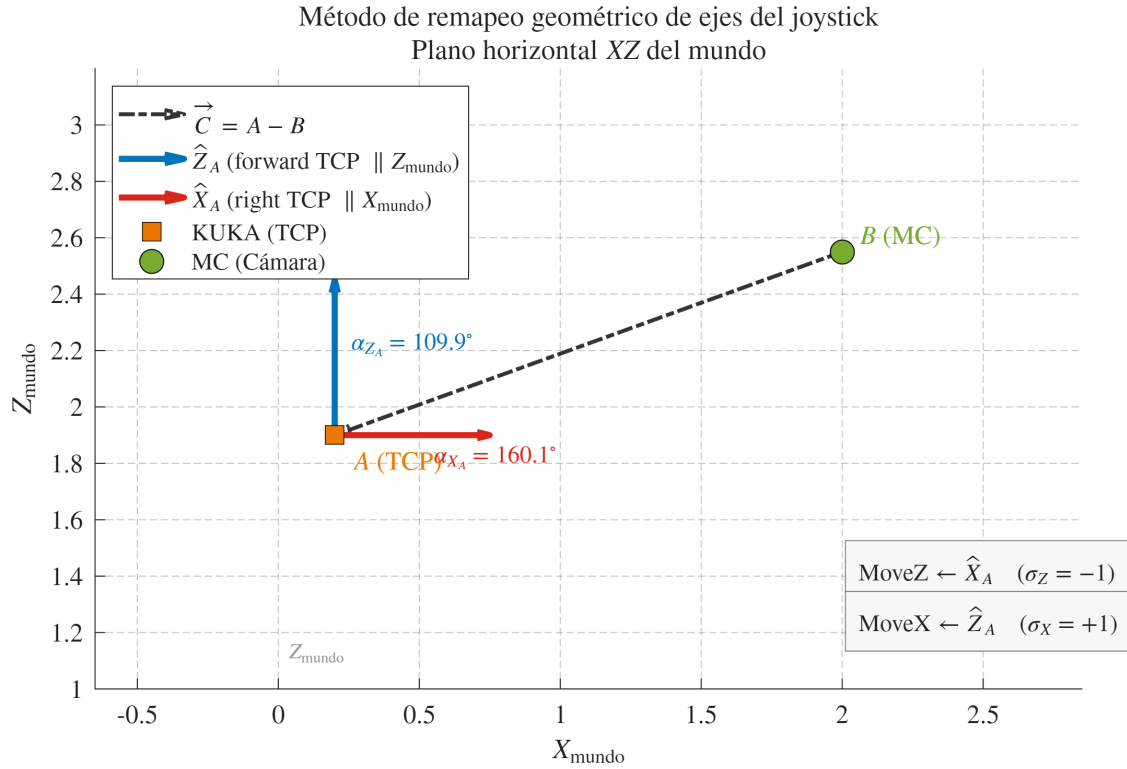


Figura 7: Plano horizontal XZ del mundo ilustrando el remapeo: el vector $\hat{\mathbf{d}}_Z$ apunta de la cámara (MC) al TCP del manipulador (KUKA), y $\hat{\mathbf{d}}_X$ se obtiene como su perpendicular en el plano horizontal.

4.3 Control PID en Articulaciones

Cuando el solver de Flange devuelve una solución IK válida, `JoystickAdapter` no aplica los ángulos resultantes directamente sobre el `MechanicalGroup`: el `JointTarget` obtenido se pasa como consigna al método `ApplyPID()`, que calcula para cada una de las seis articulaciones un torque virtual (Figura 8), lo convierte en una aceleración angular ponderada por la inercia efectiva del eslabón y finalmente integra posición y velocidad respetando límites físicos y de seguridad (Figura 14).


```
public class JoystickAdapter : MonoBehaviour
{
    private void ApplyPID(JointTarget ikTarget, float dt)

        // Calculamos la velocidad deseada para compensar perfectamente el amortiguamiento
        float qTargetVelocity = _hasPrevIkTarget && dt > 1e-6f
            ? Mathf.DeltaAngle(_prevIkTarget[i], qTarget) / dt
            : 0f;

        // PID → torque virtual (°/s² a inercia de referencia)
        float torque = _pids[i].Compute(qTarget, qActual, dt, qTargetVelocity);
        _lastJointControlActions[i] = torque;

        // Aceleración = torque / inercia normalizada → joints más pesados aceleran más lento
        float jNorm = Mathf.Max(jEff[i] / _referenceInertia, 0.05f);

        // Feedforward exacto: genera el torque necesario para mantener la velocidad venciendo el amortiguamiento
        float discreteDampingFactor = 1f / Mathf.Max(0.1f, 1f - _velocityDamping * dt);
        float feedforwardTorque = (jNorm / dt) * (qTargetVelocity * discreteDampingFactor - _jointVelocity[i]);
        torque += feedforwardTorque;

        float acceleration = torque / jNorm;
        acceleration = Mathf.Clamp(acceleration, -_maxJointAcceleration, _maxJointAcceleration);

        // Guardar para el HUD de GUI en pantalla
        _lastJointControlActions[i] = acceleration;
    }
}
```

Figura 8: Primera mitad de `ApplyPID()`: cálculo de la inercia efectiva por articulación y del torque PID con feedforward de velocidad.

La inercia efectiva de cada articulación se calcula en `RobotDynamics.ComputeEffectiveInertia()` a partir de las masas y el centro de masa (CoM) de cada eslabón del KUKA KR210, tomados del modelo URDF utilizado en el curso *Robotics Nanodegree* (RoboND) de Udacity y hardcodeados en el arreglo estático `RobotDynamics.Links`: 138 kg para `link_1`, 95 kg para `link_2`, 71 kg para `link_3`, 17 kg para `link_4`, 7 kg para `link_5` y 0.5 kg para `link_6` (Figura 9), cada uno con su CoM expresado en el espacio local del `TransformJoint` correspondiente. Para la articulación i , la inercia efectiva se obtiene como

$$J_{\text{eff}}[i] = \sum_{j \geq i} m_j \cdot d_{ij}^2 \quad (4)$$

donde d_{ij} es la distancia perpendicular entre el eje de giro de la articulación i en el mundo y el CoM (también en mundo) del eslabón j (Figura 10). Si el gripper sostiene un objeto, `ComputeEffectiveInertia()` acepta opcionalmente la masa y posición mundial del payload, obtenidos de `GripperController.GrabbedMass` y `GripperController.GrabbedWorldPosition` (Figura 11), sumándolos a $J_{\text{eff}}[i]$ de cada articulación como una masa puntual ubicada en la posición mundial del `graspPoint`.

```
Assets > Scripts > RobotDynamics.cs > RobotDynamics
1  using System.Collections.Generic;
2  using Preliy.Flange;
3  using UnityEngine;
4
5  /// <summary>
6  /// Datos de masa y CoM del KUKA KR210 R3100-2 (fuente: URDF Udacity RoboND).
7  /// Calcula la inercia efectiva por articulación según la configuración actual.
8  /// </summary>
9  0 references
10 public static class RobotDynamics
11 {
12     8 references
13     public readonly struct LinkData
14     {
15         2 references
16         public readonly float Mass; // kg
17         2 references
18         public readonly Vector3 ComLocal; // CoM en frame local del TransformJoint (m)
19
20         6 references
21         public LinkData(float mass, Vector3 comLocal)
22         {
23             Mass = mass;
24             ComLocal = comLocal;
25         }
26     }
27
28     // Índice 0 = link_1 ... índice 5 = link_6
29     // CoM expresado en el espacio local del TransformJoint correspondiente
30     2 references
31     public static readonly LinkData[] Links =
32     {
33         new LinkData(138f, new Vector3( 0.000f,  0.000f,  0.400f)), // link_1
34         new LinkData( 95f, new Vector3( 0.000f,  0.000f,  0.448f)), // link_2
35         new LinkData( 71f, new Vector3( 0.188f,  0.183f, -0.043f)), // link_3
36         new LinkData( 17f, new Vector3( 0.271f, -0.007f,  0.000f)), // link_4
37         new LinkData(  7f, new Vector3( 0.000f,  0.000f,  0.000f)), // link_5
38         new LinkData(0.5f, new Vector3( 0.000f,  0.000f,  0.000f)), // link_6
39     };
40 }
```

Figura 9: Arreglo estático RobotDynamics.Links: masa y centro de masa de cada eslabón del KUKA KR210, tomados del URDF de Udacity RoboND.

```
public static class RobotDynamics
{
    /// Devuelve  $J_{eff}[i] = \sum_{j \geq i} m_j \cdot d_{ij}^2$ 
    /// donde  $d_{ij}$  es la distancia perpendicular del eje del joint i al CoM del link j en mundo.
    ///
    /// Eje del joint i: en la convención DH de Flange todos los joints revolutos
    /// giran sobre el -Y del frame padre, por lo que el eje en mundo es
    /// i == 0 → robotJoints[0].transform.parent.up
    /// i > 0 → robotJoints[i-1].transform.up
    ///
    /// Si <paramref name="payloadMass"/> es mayor a cero, se suma como una masa puntual
    /// adicional en <paramref name="payloadWorldPos"/> (p.ej. un objeto agarrado por el
    /// gripper), sumando su propio término masa-distancia² a cada joint.
    /// </summary>
    0 references
    public static float[] ComputeEffectiveInertia(
        IReadOnlyList<TransformJoint> robotJoints,
        float payloadMass = 0f,
        Vector3? payloadWorldPos = null)
    {
        var result = new float[6];

        for (int i = 0; i < 6; i++)
        {
            Vector3 axisOrigin = robotJoints[i].transform.position;
            Vector3 axisDir = i == 0
                ? robotJoints[0].transform.parent.up
                : robotJoints[i - 1].transform.up;

            float jEff = 0f;
            for (int j = i; j < 6; j++)
            {
                Vector3 comWorld = robotJoints[j].transform.TransformPoint(Links[j].ComLocal);
                float d = PerpendicularDistance(comWorld, axisOrigin, axisDir);
                jEff += Links[j].Mass * d * d;
            }

            if (payloadMass > 0f && payloadWorldPos.HasValue)
            {
                float dPayload = PerpendicularDistance(payloadWorldPos.Value, axisOrigin, axisDir);
                jEff += payloadMass * dPayload * dPayload;
            }

            result[i] = jEff;
        }

        return result;
    }
}
```

Figura 10: Implementación de ComputeEffectiveInertia(): acumula la inercia de los eslabones distales a cada articulación y, opcionalmente, la del objeto agarrado.

```
/// <summary>Masa original (kg) del objeto actualmente agarrado, o 0 si no hay ninguno.</summary>
0 references
public float GrabbedMass => grabbedObject != null ? originalMass : 0f;

/// <summary>Posición mundial del punto de agarre (graspPoint), rígidamente ligado al objeto agarrado.</summary>
0 references
public Vector3 GrabbedWorldPosition => graspPoint != null ? graspPoint.position : transform.position;
```

Figura 11: Propiedades expuestas por GripperController para que RobotDynamics trate el objeto agarrado como masa puntual.

El controlador de cada articulación es una instancia de JointPID, con ganancias base con-

figurables en el Inspector de JoystickAdapter ($_kpBase = 10.8$, $_kiBase = 5.03$, $_kdBase = 1.03$, en unidades de $(^\circ/s^2)$ por $^\circ$, $(^\circ \cdot s)$ y $(^\circ/s)$ respectivamente, a una inercia de referencia $_referenceInertia = 5.39 \text{ kg} \cdot \text{m}^2$, Figura 12). El término derivativo no se calcula sobre la derivada clásica del error sino como un error de velocidad entre la velocidad de la consigna IK y la velocidad articular medida (Figura 13):

$$\tau[k] = K_p e[k] + K_i \sum e[k] \Delta t + K_d (\dot{q}_{\text{target}}[k] - \dot{q}_{\text{medida}}[k]) \quad (5)$$

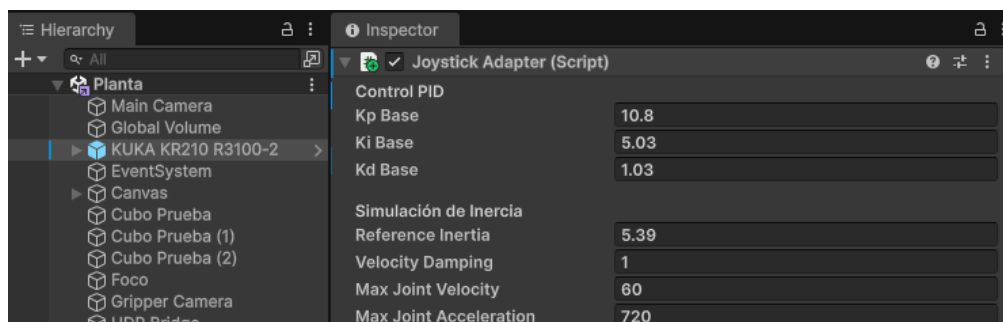


Figura 12: Componente JoystickAdapter en el Inspector: ganancias PID e inercia de referencia configuradas en la escena actual.

```

public class JointPID
{
    2 references
    public float Kp;
    2 references
    public float Ki;
    2 references
    public float Kd;
    [Tooltip("Límite anti-windup del término integral (°·s).")]
    2 references
    public float MaxIntegral = 200f;

    4 references
    private float _integral;
    3 references
    private float _prevCurrent;
    3 references
    private bool _hasPrevCurrent;

    0 references
    public JointPID(float kp, float ki, float kd)
    {
        Kp = kp;
        Ki = ki;
        Kd = kd;
    }

    /// <summary>
    /// Calcula el torque virtual para este tick.
    /// setpoint y current en grados; setpointVelocity en grados/s; dt en segundos.
    /// Retorna torque en °/s² (a inercia de referencia).
    /// </summary>
    0 references
    public float Compute(float setpoint, float current, float dt, float setpointVelocity = 0f)
    {
        float error = Mathf.DeltaAngle(current, setpoint);
        _integral = Mathf.Clamp(_integral + error * dt, -MaxIntegral, MaxIntegral);
        float measuredVelocity = _hasPrevCurrent && dt > 1e-6f
            ? Mathf.DeltaAngle(_prevCurrent, current) / dt
            : 0f;
        _prevCurrent = current;
        _hasPrevCurrent = true;
        float derivative = setpointVelocity - measuredVelocity;
        return Kp * error + Ki * _integral + Kd * derivative;
    }
}

```

Figura 13: Clase JointPID: cálculo del torque virtual por articulación, con anti-windup en el término integral.

con $e[k] = \text{Mathf.DeltaAngle}(q_{\text{actual}}, q_{\text{target}})$, término integral saturado en $\pm 200^\circ \cdot s$ (anti-windup), y $\dot{q}_{\text{medida}}$ estimada como $\text{DeltaAngle}(q_{\text{previo}}, q_{\text{actual}}) / \Delta t$. La velocidad de consigna $\dot{q}_{\text{target}}$ se estima en `JoystickAdapter` a partir de la diferencia entre el `JointTarget` IK del tick actual y el anterior, y alimenta además un término de *feedforward* que compensa el amortiguamiento viscoso discreto, garantizando que el seguimiento de velocidad no dependa únicamente de la ganancia derivativa. El torque resultante se divide por la inercia normalizada $j_{\text{norm}} = \max(J_{\text{eff}}[i] / \text{referenceInertia}, 0.05)$ —con un piso del 5% para evitar inestabilidad numérica en los eslabones livianos de la muñeca— para obtener la aceleración angular, que se satura en $\pm 720^\circ / s^2$. La velocidad se integra, se le aplica un amortiguamiento viscoso ($\text{velocityDamping} = 1 s^{-1}$) y se satura en $\pm 60^\circ / s$ (`maxJointVelocity`, salvo en J6 dentro del modo exclusivo, donde el límite es $\pm 45^\circ / s$). La posición articular resultante

se recorta contra los límites físicos de cada articulación; si un límite se alcanza, la velocidad se frena a cero y el integrador del PID correspondiente se reinicia para evitar windup. El resultado final se aplica mediante `MechanicalGroup.SetJoints()`, siempre como ángulos articulares en grados (Figura 14).

```
public class JoystickAdapter : MonoBehaviour
{
    private void ApplyPID(JointTarget ikTarget, float dt)
    {
        _jointVelocity[i] += acceleration * dt;

        // Amortiguamiento viscoso: evita aceleración indefinida
        _jointVelocity[i] -= _velocityDamping * _jointVelocity[i] * dt;

        float maxVel = (i == 5 && IsJ6ExclusiveMode) ? 45f : _maxJointVelocity;
        _jointVelocity[i] = Mathf.Clamp(_jointVelocity[i], -maxVel, maxVel);

        float targetQNew = qActual + _jointVelocity[i] * dt;
        qNew[i] = Mathf.Clamp(targetQNew, _minLimits[i], _maxLimits[i]);

        // Si se golpea un límite articular físico, frenamos la velocidad acumulada
        // en esa dirección y reseteamos el término integral del PID para evitar windup
        if (Mathf.Approximately(qNew[i], _minLimits[i]) && _jointVelocity[i] < 0f)
        {
            _jointVelocity[i] = 0f;
            _pids[i].Reset();
        }
        else if (Mathf.Approximately(qNew[i], _maxLimits[i]) && _jointVelocity[i] > 0f)
        {
            _jointVelocity[i] = 0f;
            _pids[i].Reset();
        }

        _prevIkTarget[i] = qTarget;
    }

    _hasPrevIkTarget = true;
    UpdateJointActionDisplay();
    _controller.MechanicalGroup.SetJoints(new JointTarget(qNew), notify: true);
}
```

Figura 14: Segunda mitad de `ApplyPID()`: integración de velocidad, saturaciones de seguridad y aplicación final de los ángulos articulares.

Además del control PID, `JoystickAdapter` aplica dos capas de asistencia por proximidad antes de enviar la trayectoria al solver IK, ambas alimentadas por el sensor inferior de `GripperDistanceSensor` (Figura 15). La referencia a ese sensor (`_proximitySensor`) puede dejarse sin asignar en el Inspector: si es así, `Awake()` la resuelve automáticamente buscando en la escena la primera instancia de `GripperDistanceSensor` con `ContributesToSpeedReduction` activo, evitando así una referencia manual propensa a errores si se reorganiza la jerarquía del gripper. La primera capa es un frenado progresivo de la velocidad cartesiana completa, en cualquier dirección de movimiento: a medida que la distancia detectada cae por debajo de un umbral configurable (`ProximitySlowdownSettings.ThresholdMeters`, 30 cm por defecto, Figura 16), la velocidad se interpola linealmente hasta un

piso mínimo (`_proximitySpeedMultiplier`, 50% por defecto), sin llegar nunca a cero. La segunda, `ApplyDescentLimit()`, es un bloqueo estricto que solo actúa al descender con la garra cerrada: recorta el paso vertical al margen de seguridad restante (`ProximitySlowdownSettings.DescentMarginMeters`, 5 cm por defecto), deteniendo el descenso justo sobre la superficie en lugar de dejar que el gripper la penetre (Figura 17). Ambos umbrales son ajustables por el operador en tiempo real desde el menú de pausa, ciclando entre presets predefinidos, y se persisten entre sesiones en `PlayerPrefs` (Figura 18).

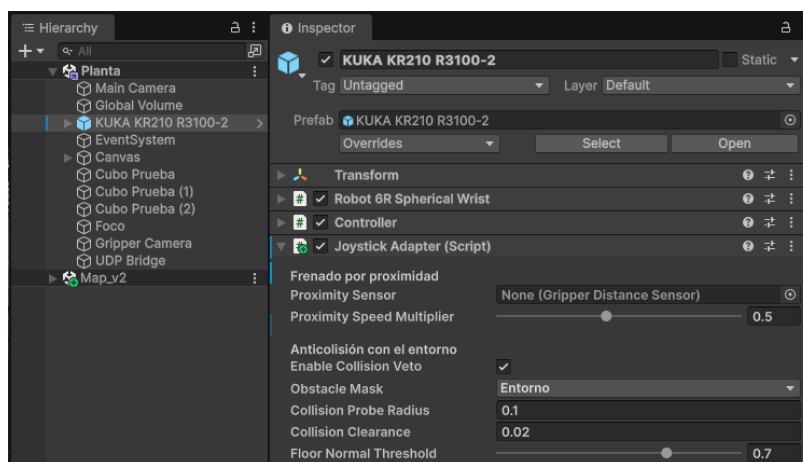


Figura 15: Componente `JoystickAdapter`: secciones *Frenado por proximidad* y *Anticolisión con el entorno*. El campo `Proximity Sensor` se deja sin asignar a propósito, ya que `Awake()` lo resuelve automáticamente.

```
public static class ProximitySlowdownSettings
{
    2 references
    private const string ThresholdPrefsKey = "ProximitySlowdown.ThresholdMeters.v2";
    2 references
    private const string DescentMarginPrefsKey = "ProximitySlowdown.DescentMarginMeters";
    2 references
    private const float DefaultThresholdMeters = 0.30f;
    2 references
    private const float DefaultDescentMarginMeters = 0.05f;

    /// <summary>Umbral de frenado seleccionable desde el menu de pausa, en metros. 0 = sin frenado.</summary>
    3 references
    public static readonly float[] Presets = { 0.10f, 0.20f, 0.30f, 0.40f, 0.50f, 0f };

    /// <summary>Margen de bloqueo de descenso seleccionable desde el menu de pausa, en metros. 0 = sin bloqueo.</summary>
    3 references
    public static readonly float[] DescentMarginPresets = { 0.03f, 0.05f, 0.08f, 0.12f, 0f };

    2 references
    private static bool _loaded;
    6 references
    private static float _thresholdMeters = DefaultThresholdMeters;
    6 references
    private static float _descentMarginMeters = DefaultDescentMarginMeters;

    /// <summary>Se dispara cuando el operario cambia cualquiera de los dos ajustes.</summary>
    2 references
    public static event System.Action ThresholdChanged;

    3 references
    public static float ThresholdMeters
    {
        get
        {
            EnsureLoaded();
            return _thresholdMeters;
        }
    }

    3 references
    public static float DescentMarginMeters
    {
        get
        {
            EnsureLoaded();
            return _descentMarginMeters;
        }
    }

    1 reference
    public static bool IsEnabled => ThresholdMeters > 0f;

    1 reference
    public static bool IsDescentBlockEnabled => DescentMarginMeters > 0f;
}
```

Figura 16: Clase estática ProximitySlowdownSettings: centraliza los umbrales de frenado y bloqueo de descenso, seleccionables desde el menú de pausa y persistidos en PlayerPrefs.

```
public class JoystickAdapter : MonoBehaviour
{
    /// <summary>
    /// Con la garra cerrada, impide seguir bajando por debajo del margen de seguridad configurado.
    /// Subir nunca se limita. Se aplica en coordenadas de mundo y ANTES de WorldVectorToFrame porque
    /// la componente vertical del input se compone como Vector3.up * rawY, también en mundo.
    /// El paso se recorta al margen restante en vez de cortarse en seco, para que el brazo se detenga
    /// justo sobre el margen en lugar de pasarse un tick.
    /// </summary>
    1 reference
    private void ApplyDescentLimit(ref Vector3 deltaWorld)
    {
        IsDescentBlocked = false;

        if (deltaWorld.y >= 0f) return;
        if (!ProximitySlowdownSettings.IsDescentBlockEnabled) return;
        if (_gripperController == null || !_gripperController.IsGripperClosed) return;
        if (_proximitySensor == null || !_proximitySensor.HasHit) return;

        float allowedDescent = Mathf.Max(
            0f,
            _proximitySensor.Distance - ProximitySlowdownSettings.DescentMarginMeters);

        if (deltaWorld.y >= -allowedDescent) return;

        deltaWorld.y = -allowedDescent;
        IsDescentBlocked = true;
    }
}
```

Figura 17: ApplyDescentLimit(): recorta el descenso vertical al margen de seguridad restante cuando la garra está cerrada.

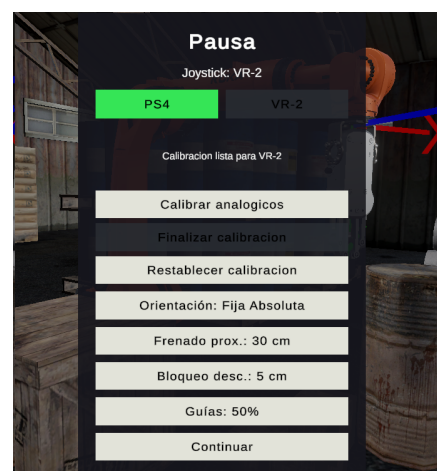
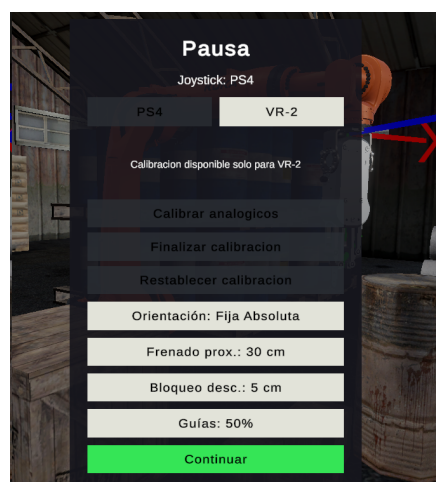


Figura 18: Menú de pausa (PauseMenuController) para los dos perfiles de joystick soportados: PS4 (izquierda) y VR-2 (derecha), con los botones *Frenado prox.* y *Bloqueo desc.* que ciclan entre los presets configurados.

```
public class JoystickAdapter : MonoBehaviour
{
    private void ApplyCollisionVeto(ref Vector3 deltaWorld)
    {
        IsMotionBlocked = false;

        if (!_enableCollisionVeto || _endEffector == null) return;

        float stepDistance = deltaWorld.magnitude;
        if (stepDistance <= 1e-6f) return;

        Vector3 direction = deltaWorld / stepDistance;
        float castDistance = stepDistance + _collisionClearance;

        int hitCount = Physics.SphereCastNonAlloc(
            _endEffector.position,
            _collisionProbeRadius,
            direction,
            _collisionBuffer,
            castDistance,
            _obstacleMask,
            QueryTriggerInteraction.Ignore);

        float nearest = float.MaxValue;
        for (int hitIndex = 0; hitIndex < hitCount; hitIndex++)
        {
            RaycastHit hit = _collisionBuffer[hitIndex];

            // Solapamiento en el origen: Unity devuelve distancia 0. Si lo tomásemos como obstáculo,
            // el brazo quedaría congelado sin forma de salir. Se ignora para que el operador pueda
            // retroceder.
            if (hit.distance <= 1e-4f) continue;
            if (!IsObstacle(hit.collider)) continue;

            // Suelos y superficies horizontales: las gobierna el bloqueo de descenso, que es preciso
            // y tiene en cuenta la pieza transportada. Vetarlas aquí impediría bajar a recoger.
            if (Vector3.Dot(hit.normal, Vector3.up) > _floorNormalThreshold) continue;

            if (hit.distance < nearest)
            {
                nearest = hit.distance;
            }
        }

        if (nearest == float.MaxValue) return;

        float allowedStep = Mathf.Max(0f, nearest - _collisionClearance);
        if (allowedStep >= stepDistance) return;

        deltaWorld = direction * allowedStep;
        IsMotionBlocked = true;
    }

    /// <summary>
    /// Descarta el propio robot y la pieza agarrada (que al agarrarse pasa a colgar del gripper).
    /// </summary>
    1 reference
    private bool IsObstacle(Collider candidate)
    {
        if (candidate == null || !candidate.enabled) return false;

        return _robotRoot == null || !candidate.transform.IsChildOf(_robotRoot);
    }
}
```

Figura 19: `ApplyCollisionVeto()`: recorta el paso cartesiano del efector antes de tocar un obstáculo del entorno, ignorando el propio robot, la pieza agarrada y las superficies muy horizontales.

El robot cuenta además con un veto anticolidión con el entorno, independiente de los dos mecanismos anteriores: `ApplyCollisionVeto()` traza un `SphereCastNonAlloc` desde el efector final a lo largo de la dirección de movimiento contra el *layer* Entorno, descartando el propio robot y la pieza agarrada (mediante `IsObstacle()`), y recorta el paso cartesiano antes de que el efector llegue a tocar un obstáculo (Figura 19). El veto ignora deliberadamente superficies muy horizontales —ya gobernadas por el bloqueo de descenso— para no impedir que el operador baje a recoger una pieza apoyada en una mesa o estante. Este mecanismo depende de colliders generados sobre la geometría del entorno mediante `Tools > Entorno > Generar colliders faltantes` en el editor; sin ese paso de configuración manual, el veto queda sin efecto.

4.4 Movimiento del Gripper y Realimentación Háptica

La apertura y cierre de la garra se controla con un único botón (acción Gripper, R3), gestionado por `Ctrl_OnRobotRG2_Custom`. Un clic corto alterna el estado `_isOpen` y dispara la animación del efector OnRobot RG2: la carrera (*stroke*, 0–100 mm) se convierte en un ángulo de servo mediante un polinomio de ajuste (*Polyval*) y se interpola con `Mathf.MoveTowards` a una velocidad configurable por Inspector, distinta para apertura y cierre (*Open Speed* = 120 mm/s, *Close Speed* = 140 mm/s, Figura 22), rotando físicamente los seis *Rigidbody* de los dedos mediante *MoveRotation* (Figura 21). Mantener el botón presionado un segundo, en cambio, no cierra la garra sino que dispara un *homing* de la articulación J6 a su cero físico de 17.7° (`JoystickAdapter.ResetJ6ToZero()`), la misma acción asociada al botón Cuadrado (*J6Home*, Figura 20).

```
public class Ctrl_OnRobotRG2_Custom : MonoBehaviour
{
    private void OnToggleGripStarted(InputAction.CallbackContext ctx)
    {
        if (_inputSuppressed)
            return;

        _isHolding = true;
        _pressTime = Time.unscaledTime;
        _homingTriggered = false;
    }

    2 references
    private void OnToggleGripCanceled(InputAction.CallbackContext ctx)
    {
        if (_inputSuppressed)
            return;

        _isHolding = false;

        // Si se soltó antes del umbral de hold, fue un clic normal
        if (!_homingTriggered)
        {
            _isOpen = !_isOpen;
            Debug.Log($"[Gripper] Toggle → _isOpen={_isOpen}, stroke={{(_isOpen ? s_max : s_min)}}");
            stroke = _isOpen ? s_max : s_min;
            speed = _isOpen ? _openSpeed : _closeSpeed;
            start_movement = true;

            if (_grripperController != null)
            {
                _grripperController.ToggleGrip();
            }
        }
    }

    0 references | ☺ Unity Message
    private void Update()
    {
        if (_isHolding && !_homingTriggered && (Time.unscaledTime - _pressTime >= 1.0f))
        {
            _homingTriggered = true;
            Debug.Log("[Gripper] Hold detected for 1 sec! Triggering J6 Homing.");

            var joystickAdapter = FindFirstObjectByType<JoystickAdapter>();
            if (joystickAdapter != null)
            {
                joystickAdapter.ResetJ6ToZero();
            }
        }
    }
}
```

Figura 20: Distinción entre pulsación corta (toggle de apertura/cierre) y mantenida 1 s (homing de J6) en `Ctrl_OnRobotRG2_Custom`.

```
public class Ctrl_OnRobotRG2_Custom : MonoBehaviour
{
    void FixedUpdate()
    {
        case 1:
        {
            // Reset variables.
            in_position = false;

            // Convert the stroke to the angle in degrees.
            __theta = Polyval(coefficients, __stroke) * Mathf.Rad2Deg;

            ctrl_state = 2;
        }
        break;

        case 2:
        {
            // Interpolate the orientation between the current position and the target position.
            __theta_i = Mathf.MoveTowards(__theta_i, __theta, __speed * Time.deltaTime);

            // Change the orientation of the end-effector arm.
            Quaternion rotR0 = Quaternion.Euler(0.0f, -__theta_i, 0.0f);
            Quaternion rotR1 = Quaternion.Euler(0.0f, -__theta_i, 0.0f);
            Quaternion rotR2 = Quaternion.Euler(0.0f, -__theta_i, 0.0f);

            Quaternion rotL0 = Quaternion.Euler(0.0f, __theta_i, 0.0f);
            Quaternion rotL1 = Quaternion.Euler(0.0f, __theta_i, 0.0f);
            Quaternion rotL2 = Quaternion.Euler(0.0f, -__theta_i, 0.0f);

            // Right arm.
            if (rb_R_Arm_ID_0 != null) rb_R_Arm_ID_0.MoveRotation(R_Arm_ID_0.transform.parent.rotation * rotR0);
            else R_Arm_ID_0.transform.localRotation = rotR0;

            if (rb_R_Arm_ID_1 != null) rb_R_Arm_ID_1.MoveRotation(R_Arm_ID_1.transform.parent.rotation * rotR1);
            else R_Arm_ID_1.transform.localRotation = rotR1;

            if (rb_R_Arm_ID_2 != null) rb_R_Arm_ID_2.MoveRotation(R_Arm_ID_2.transform.parent.rotation * rotR2);
            else R_Arm_ID_2.transform.localRotation = rotR2;

            // Left arm.
            if (rb_L_Arm_ID_0 != null) rb_L_Arm_ID_0.MoveRotation(L_Arm_ID_0.transform.parent.rotation * rotL0);
            else L_Arm_ID_0.transform.localRotation = rotL0;

            if (rb_L_Arm_ID_1 != null) rb_L_Arm_ID_1.MoveRotation(L_Arm_ID_1.transform.parent.rotation * rotL1);
            else L_Arm_ID_1.transform.localRotation = rotL1;

            if (rb_L_Arm_ID_2 != null) rb_L_Arm_ID_2.MoveRotation(L_Arm_ID_2.transform.parent.rotation * rotL2);
            else L_Arm_ID_2.transform.localRotation = rotL2;

            if (__theta_i == __theta)
            {
                in_position = true; start_movement = false;
                ctrl_state = 0;
            }
        }
    }
}
```

Figura 21: Máquina de estados de Ctrl_OnRobotRG2_Custom: conversión de carrera a ángulo y aplicación física sobre los Rigidbody de los dedos.

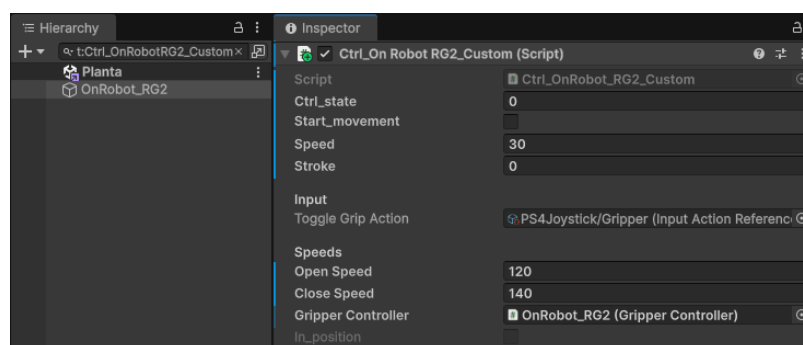


Figura 22: Configuración del componente Ctrl_OnRobotRG2_Custom: acción de entrada (R3) y velocidades de apertura/cierre.

La lógica de agarre vive en **GripperController** y no depende únicamente de un tag ni de un simple raycast: cada dedo tiene un **GripperTriggerForwarder** sobre su cara interna (Figura 25) que detecta por *trigger* los objetos con tag "Agarrable" recorriendo la jerarquía de **Transform** hacia arriba desde el colisionador detectado (Figura 24) y notifica los contactos a **GripperController**.

`NotifyFingerContact()`. Un objeto solo se agarra si existe una ventana de cierre explícita (`isClosing`) y si el objeto registra contacto simultáneo de ambos dedos (`HasOpposingInnerContacts()`, Figura 23); tocar un objeto con la garra ya cerrada nunca lo adhiere. Al agarrarlo, `GrabObject()` guarda la masa original del `Rigidbody`, la suma a la masa del `Rigidbody` del gripper, vuelve cinemático el objeto agarrado y lo emparenta al `graspPoint` para lograr un movimiento 1:1 sin deslizamientos (Figura 26). Al soltar, se restauran masa y físicas, y `MoveGrabbedObjectToSafeReleaseHeight()` corrige la posición si la suelta ocurriera por debajo del piso o de la altura mínima de recogida (Figura 27).

```
public class GripperController : MonoBehaviour
{
    private void TryGrab()
    {
        if (grabbedObject != null || !isClosing) return;

        GameObject objectToGrab = null;
        foreach (KeyValuePair<GameObject, HashSet<GripperTriggerForwarder>> entry in fingerContacts)
        {
            if (HasOpposingInnerContacts(entry.Value))
            {
                objectToGrab = entry.Key;
                break;
            }
        }

        if (objectToGrab != null)
        {
            GrabObject(objectToGrab);
        }
    }

    1 reference
    private bool HasOpposingInnerContacts(HashSet<GripperTriggerForwarder> contacts)
    {
        bool touchesLeft = false;
        bool touchesRight = false;

        foreach (GripperTriggerForwarder contact in contacts)
        {
            if (contact == null || !contact.IsInnerFace) continue;

            switch (contact.ResolveFingerSide(transform))
            {
                case GripperFingerSide.Left:
                    touchesLeft = true;
                    break;
                case GripperFingerSide.Right:
                    touchesRight = true;
                    break;
            }

            if (touchesLeft && touchesRight) return true;
        }

        return false;
    }
}
```

Figura 23: Lógica de agarre en `GripperController`: solo se agarra un objeto con contacto simultáneo de ambos dedos y ventana de cierre activa.

```
public class GripperTriggerForwarder : MonoBehaviour
{
    private static bool TryGetGrabbableObject(Collider other, out GameObject grabbable)
    {
        grabbable = null;
        if (other == null) return false;

        Transform current = other.transform;
        while (current != null)
        {
            if (current.CompareTag("Agarrable"))
            {
                grabbable = other.attachedRigidbody != null
                    ? other.attachedRigidbody.gameObject
                    : current.gameObject;
                return true;
            }

            current = current.parent;
        }

        return false;
    }
}
```

Figura 24: Método TryGetGrabbableObject de GripperTriggerForwarder: resuelve el objeto agarrable subiendo por la jerarquía del Transform hasta encontrar el tag Agarrable.

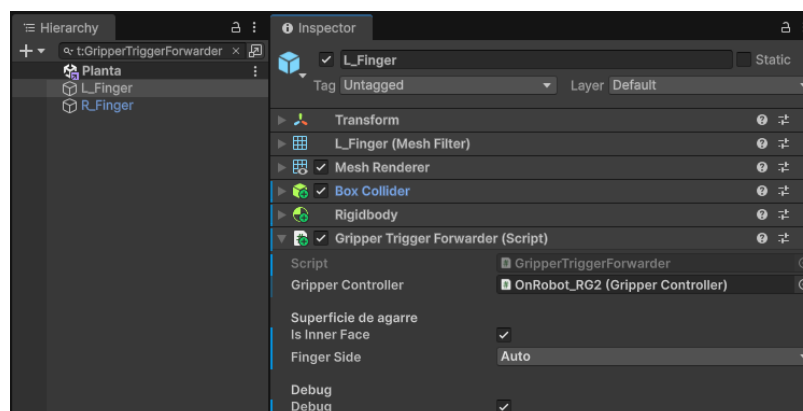


Figura 25: Componente GripperTriggerForwarder en la cara interna de uno de los dedos del gripper.

```
public class GripperController : MonoBehaviour
{
    private void GrabObject(GameObject objectToGrab)
    {
        isClosing = false;
        grabbedObject = objectToGrab;
        grabbedRigidbody = grabbedObject.GetComponent<Rigidbody>();
        Physics.SyncTransforms();
        pickupMinimumY = GetLowestSolidPoint(grabbedObject);

        if (grabbedRigidbody != null)
        {
            // Guardar masa original y transferirla al gripper
            originalMass = grabbedRigidbody.mass;
            if (gripperRigidbody != null)
            {
                gripperRigidbody.mass += originalMass;
            }

            // Hacer el objeto cinemático para evitar deslizamientos
            grabbedRigidbody.isKinematic = true;
        }

        // Emparentar para movimiento 1:1 perfecto
        Transform parent = graspPoint != null ? graspPoint : transform;
        grabbedObject.transform.SetParent(parent);

        if (debugTriggers) Debug.Log($"[GripperController] GrabObject: grabbed {grabbedObject.name} (Mass: {originalMass} transferred to gripper)");

        if (gripperAnimator != null)
        {
            gripperAnimator.start_movement = false;
            gripperAnimator.ctrl_state = 0;
            gripperAnimator.in_position = true;
            if (debugTriggers) Debug.Log($"[GripperController] Stopped gripper animator movement");
        }
    }
}
```

Figura 26: GrabObject(): transferencia de masa al gripper, cinemática del objeto y emparentado al graspPoint.

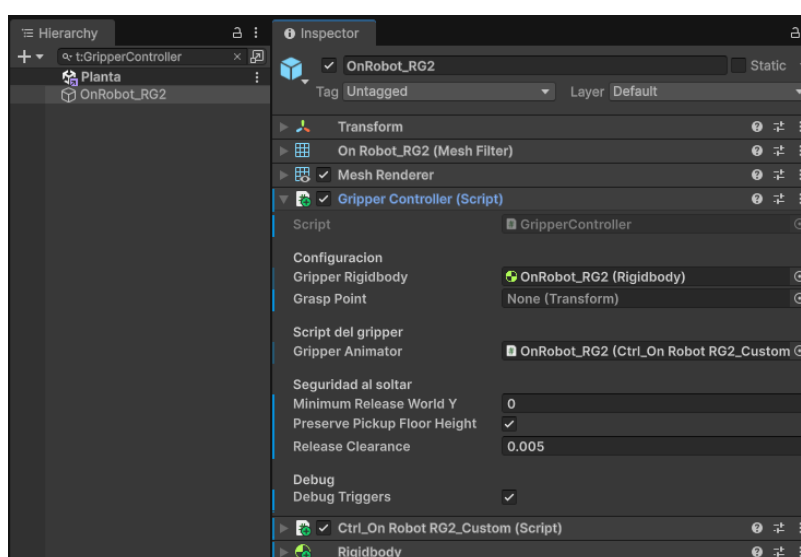


Figura 27: Componente GripperController: referencias y parámetros de seguridad al soltar un objeto.

La proximidad para la realimentación háptica y para el frenado por proximidad (Sección 4.3 bis / Extras) se mide con cinco instancias de **GripperDistanceSensor**, cada una con un rol distinto. El sensor inferior opera en modo *Sphere Cast* —un rayo con grosor de 0.01 m trazado a lo largo de un único eje— y es el único que alimenta el HUD de agarre seguro (**IsGripDistanceSafe**, entre 0.01 y 0.05 m) y el frenado por proximidad del brazo, ya que exige una lectura puntual y precisa justo debajo del gripper (Figura 28). Cuatro sensores laterales adicionales (**Sensor Distancia Derecho/Izquierdo**) operan en modo *Cono*, con un muestreo de **Physics.OverlapSphereNonAlloc** (8 muestras, 22.5°

de semiángulo, hasta 0.4 m) que promedia sobre un área en lugar de un único punto (Figura 29); no participan del frenado ni del HUD (Figura 30), y su única función es disparar el motor háptico de forma independiente ante un obstáculo cercano en su dirección, como aviso temprano de colisión lateral durante el traslado del TCP. Independientemente de ese umbral de seguridad, cuando no hay ningún objeto agarrado y la distancia detectada cae por debajo de `vibrationStartDistance` (0.15 m, con histéresis hasta 0.18 m para desactivar), cualquiera de los cinco sensores puede activar la vibración del joystick a través de `JoystickVibrationHidOutput.SetMotor(true)`, de forma independiente entre sí (Figura 31).

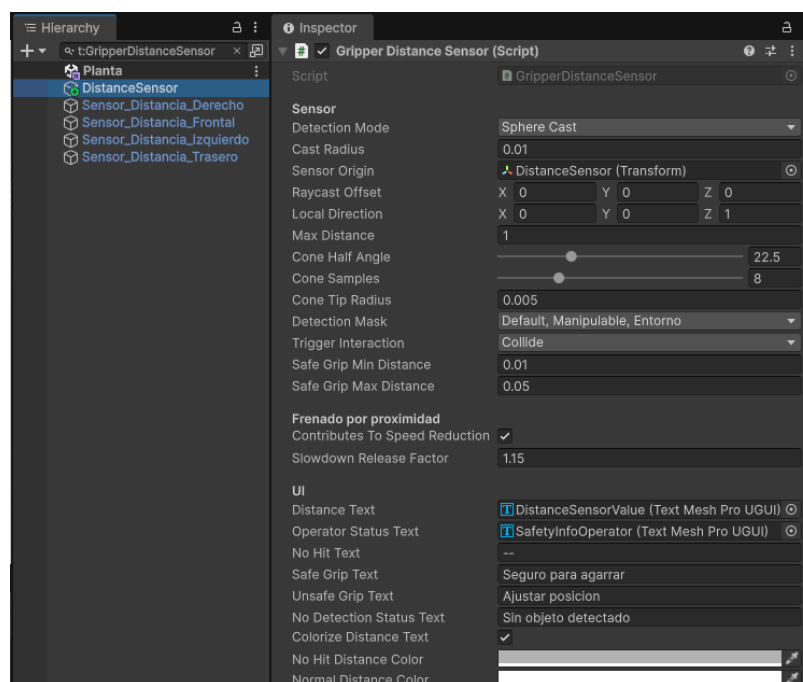


Figura 28: `GripperDistanceSensor` del sensor inferior: modo *Sphere Cast*, único que alimenta el HUD de agarre seguro y el frenado por proximidad.

```
public class GripperDistanceSensor : MonoBehaviour
{
    private void MeasureWithCone(Vector3 startPoint, Vector3 direction)
    {
        float coneSlope = Mathf.Tan(coneHalfAngle * Mathf.Deg2Rad);
        for (int sampleIndex = 1; sampleIndex <= coneSamples; sampleIndex++)
        {
            float sampleRatio = sampleIndex / (float)coneSamples;
            float axialDistance = maxDistance * sampleRatio;
            float sampleRadius = coneTipRadius + axialDistance * coneSlope;
            Vector3 sampleCenter = startPoint + direction * axialDistance;

            int overlapCount = Physics.OverlapSphereNonAlloc(
                sampleCenter,
                sampleRadius,
                overlapBuffer,
                detectionMask,
                triggerInteraction);

            for (int colliderIndex = 0; colliderIndex < overlapCount; colliderIndex++)
            {
                Collider candidate = overlapBuffer[colliderIndex];
                if (!CanDetect(candidate)) continue;

                Vector3 closestPoint = candidate.ClosestPoint(sampleCenter);
                Vector3 fromOrigin = closestPoint - startPoint;
                float candidateAxialDistance = Vector3.Dot(fromOrigin, direction);
                if (candidateAxialDistance < 0f || candidateAxialDistance > maxDistance) continue;

                Vector3 radialVector = fromOrigin - direction * candidateAxialDistance;
                float allowedRadius = coneTipRadius + candidateAxialDistance * coneSlope;
                if (radialVector.sqrMagnitude > allowedRadius * allowedRadius) continue;

                float candidateDistance = fromOrigin.magnitude;
                if (HasHit && candidateDistance >= Distance) continue;

                HasHit = true;
                Distance = candidateDistance;
                DetectedCollider = candidate;
                DetectionPoint = closestPoint;
            }
        }
    }
}
```

Figura 29: Método MeasureWithCone() de GripperDistanceSensor: muestreo cónico mediante OverlapSphereNonAlloc, usado por los cuatro sensores laterales.

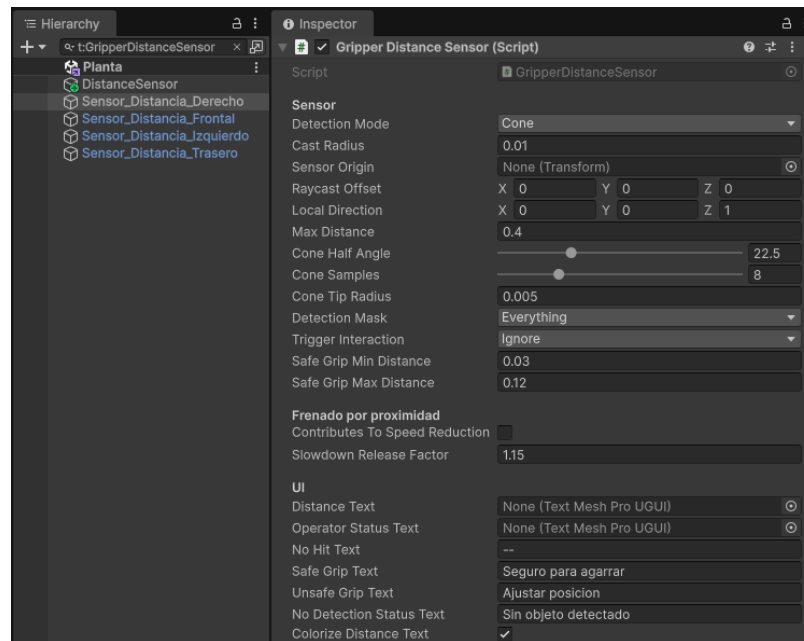


Figura 30: GripperDistanceSensor de uno de los cuatro sensores laterales: modo *Cono*, sin HUD ni participación en el frenado por proximidad.

```
public class GripperDistanceSensor : MonoBehaviour
{
    private void UpdateJoystickVibration()
    {
        if (!vibrateOnProximity || joystickVibration == null)
        {
            SetProximityVibration(false);
            return;
        }

        // Si ya tenemos un objeto agarrado, no debemos vibrar
        if (gripperController != null && gripperController.IsHoldingObject)
        {
            SetProximityVibration(false);
            return;
        }

        float threshold = IsVibratingForProximity
            ? vibrationStopDistance
            : vibrationStartDistance;

        bool shouldVibrate = HasHit && Distance <= threshold;
        SetProximityVibration(shouldVibrate);
    }

    4 references
    private void SetProximityVibration(bool enabled)
    {
        if (IsVibratingForProximity == enabled) return;

        IsVibratingForProximity = enabled;
        if (enabled)
            VibratingSensors.Add(this);
        else
            VibratingSensors.Remove(this);

        joystickVibration?.SetMotor(VibratingSensors.Count > 0);
    }
}
```

Figura 31: UpdateJoystickVibration() y SetProximityVibration(): activación de la vibración por histéresis, independiente en cada una de las cinco instancias.

El envío del comando de vibración desde Unity al joystick RPi Pico opera por dos caminos en paralelo dentro de JoystickVibrationHidOutput. El principal es una escritura HID directa por Win32 (CreateFile/WriteFile/HidD_SetOutputReport sobre hid.dll y setupapi.dll), localizando el dispositivo por su VID/PID (0xCAFE/0x4004, Figura 34) y enviando un reporte de salida de dos bytes: un *report ID* (0x00) seguido de un valor de motor (0x01 encendido, 0x00 apagado, Figura 32). En paralelo, si hay un gamepad PS4 conectado a través del Input System (DualShockGamepad), se replica el estado del motor como *rumble* nativo (SetMotorSpeeds), lo que permite sustituir el joystick RPi Pico por un control PS4 estándar durante sesiones de prueba sin hardware (Figura 33).


```
public class JoystickVibrationHidOutput : MonoBehaviour
{
    private bool TrySendRequestedState(bool force = false)
    {
        if (!force && _sentState.HasValue && _sentState.Value == _requestedState)
            return true;

        if (!EnsureDevice())
            return false;

        byte reportId = (byte)Mathf.Clamp(reportIdPrefix, 0, 255);
        byte value = (byte)(_requestedState ? motorOnValue : motorOffValue);

        if (_device.WriteOutputReport(reportId, value))
        {
            _sentState = _requestedState;
            _loggedWriteFailure = false;
            return true;
        }

        if (logConnection && !_loggedWriteFailure)
        {
            Debug.LogWarning("[JoystickVibrationHidOutput] No se pudo escribir el reporte HID del motor.");
            _loggedWriteFailure = true;
        }

        CloseDevice();
        ScheduleReconnect();
        return false;
    }
}
```

Figura 32: TrySendRequestedState(): escritura del reporte HID de dos bytes hacia el joystick RPi Pico, con reconexión automática si falla.

```
public class JoystickVibrationHidOutput : MonoBehaviour
{
    private bool TrySendPs4RequestedState(bool force = false)
    {
        if (!vibratePs4Gamepads)
        {
            _sentPs4State = null;
            return false;
        }

        if (!force && Time.unscaledTime < _nextPs4RefreshTime)
        {
            return _sentPs4State.HasValue && _sentPs4State.Value == _requestedState;
        }

        bool needsRefresh = _requestedState && Time.unscaledTime >= _nextPs4RefreshTime;
        if (!force && _sentPs4State.HasValue && _sentPs4State.Value == _requestedState && !needsRefresh)
            return true;

        float lowFrequency = _requestedState ? ps4LowFrequencyMotor : 0f;
        float highFrequency = _requestedState ? ps4HighFrequencyMotor : 0f;
        bool foundPs4Gamepad = false;

        foreach (InputDevice device in InputSystem.devices)
        {
            if (device is not DualShockGamepad gamepad) continue;

            gamepad.SetMotorSpeeds(lowFrequency, highFrequency);
            foundPs4Gamepad = true;
        }

        _sentPs4State = foundPs4Gamepad ? _requestedState : null;
        _nextPs4RefreshTime = Time.unscaledTime + Mathf.Max(0.5f, ps4RumbleRefreshInterval);
        return foundPs4Gamepad;
    }
}
```

Figura 33: TrySendPs4RequestedState(): camino alternativo de vibración, replicando el estado del motor como *rumble* nativo en un gamepad PS4 conectado.

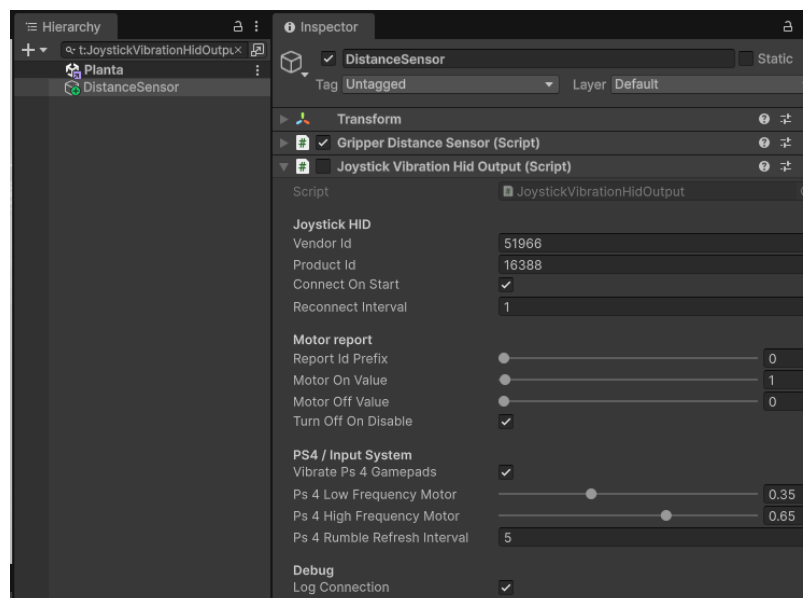


Figura 34: Componente JoystickVibrationHidOutput: identificación del dispositivo HID y parámetros de *rumble* PS4.

La cámara del gripper se implementa como una **Camera** independiente con **targetTexture** apun-

tando a un `RenderTexture` dedicado, mostrado en el Canvas mediante una `RawImage` en la esquina inferior izquierda (Figura 35). Su posición y orientación se actualizan en `GripperTopCameraFollow`. `LateUpdate()`: la cámara sigue al efector final desde arriba y se mantiene siempre mirando hacia abajo, con el vector *up* proyectado desde el *forward* del efector sobre el plano horizontal, de modo que la vista gira solidariamente con J6 (Figura 36).

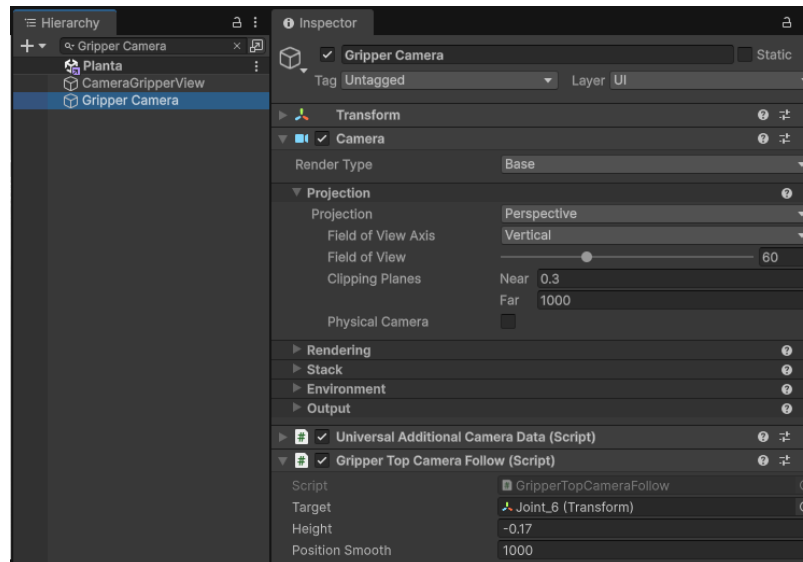


Figura 35: GameObject `Gripper Camera`: cámara dedicada con `RenderTexture` de salida y script de seguimiento del efector.

```
> Scripts > C# GripperTopCameraFollow.cs > ...
using UnityEngine;

0 references | ② Unity Script (9 asset references)
public class GripperTopCameraFollow : MonoBehaviour
{
    4 references | ② Unity Serialized Field = KUKA KR210 R3100-2
    [SerializeField] private Transform target;
    1 reference | ② Unity Serialized Field = -0.17
    [SerializeField] private float height = -0.17f;
    1 reference | ② Unity Serialized Field = 1000
    [SerializeField] private float positionSmooth = 1000f;

    0 references | ② Unity Message
    private void LateUpdate()
    {
        if (target == null) return;

        Vector3 desiredPosition = target.position + Vector3.up * height;
        transform.position = Vector3.Lerp(
            transform.position,
            desiredPosition,
            Time.deltaTime * positionSmooth
        );

        // Proyectar el vector forward del efector en el plano horizontal para usarlo como 'up' de la cámara
        Vector3 targetForward = target.forward;
        targetForward.y = 0f;

        if (targetForward.sqrMagnitude < 1e-6f)
        {
            targetForward = target.up;
            targetForward.y = 0f;
        }

        Vector3 cameraUp = targetForward.sqrMagnitude > 1e-6f ? targetForward.normalized : Vector3.forward;

        // La cámara siempre mira hacia abajo (LookRotation(down, up)) pero rota con J6
        transform.rotation = Quaternion.LookRotation(Vector3.down, cameraUp);
    }
}
```

Figura 36: Script GripperTopCameraFollow completo: la cámara del gripper sigue al efector y gira solidariamente con J6.

4.5 Comunicación UDP con Arquitectura Servidor–Cliente

La aplicación Unity actúa como **servidor UDP** con arquitectura de suscripción dinámica, implementada en el script `JointStateBroadcaster.cs`. A diferencia de un esquema de destino fijo, el servidor mantiene una lista de suscriptores activos: cualquier cliente que envíe un datagrama de suscripción ("HELLO") a la IP y puerto de Unity queda registrado y comienza a recibir el estado articular del robot (Figura 38). El servidor soporta opcionalmente la eliminación automática de suscriptores inactivos (`_autoCleanupEnabled`, desactivado por defecto) tras un tiempo de inactividad configurable (`_subscriberTimeoutSeconds = 15 s`, Figura 39); cuando está activado, los clientes que dejan de enviar *keep-alive* periódicos son removidos de la lista. Esto permite que múltiples clientes —un script de Python, una visualización en MATLAB, un dashboard web en el celular— operen simultáneamente sin ninguna reconfiguración del servidor. El HUD del operador incluye además un panel "*Cientes conectados*" (`ConnectedClientsPanel.cs`) que lista en pantalla, en vivo, las IPs actualmente suscriptas (Figura 37).

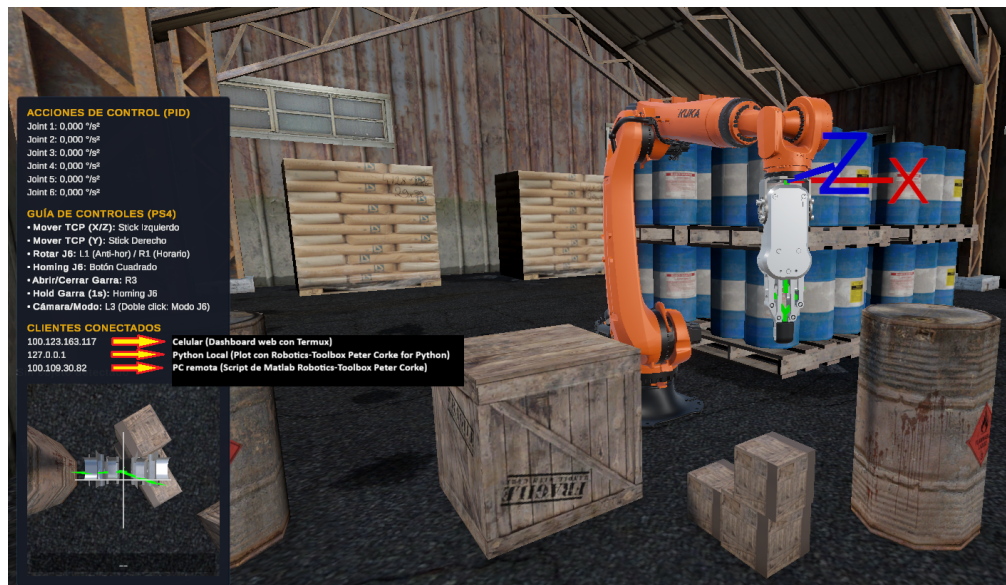


Figura 37: HUD del operador en Play Mode: panel "*Cientes conectados*" listando en vivo las IPs suscriptas al servidor UDP.

```
public class JointStateBroadcaster : MonoBehaviour
{
    private void RemoveInactiveSubscribers_NoLock()
    {
        double now = _stopwatch.Elapsed.TotalSeconds;
        List<IPEndPoint> expired = null;

        foreach (var kvp in _subscribers)
        {
            if (now - kvp.Value > _subscriberTimeoutSeconds)
            {
                expired ??= new List<IPEndPoint>();
                expired.Add(kvp.Key);
            }
        }

        if (expired == null) return;
        foreach (var endPoint in expired)
            _subscribers.Remove(endPoint);
    }

    3 references
    private void BeginListening(UdpClient client)
    {
        if (_isClosing || client == null) return;

        try
        {
            client.BeginReceive(OnReceive, client);
        }
        catch (ObjectDisposedException)
        {
        }
        catch (SocketException)
        {
        }
    }

    1 reference
    private void OnReceive(IAsyncResult asyncResult)
    {
        if (_isClosing) return;

        var client = (UdpClient)asyncResult.AsyncState;
        IPEndPoint remoteEndPoint = null;

        try
        {
            client.EndReceive(asyncResult, ref remoteEndPoint);
        }
        catch (ObjectDisposedException)
        {
            return;
        }
        catch (SocketException)
        {
            BeginListening(client);
            return;
        }

        double now = _stopwatch.Elapsed.TotalSeconds;
        lock (_subscribersLock)
        {
            _subscribers[remoteEndPoint] = now;
        }

        BeginListening(client);
    }
}
```

Figura 38: JointStateBroadcaster: registro de suscriptores por datagrama "HELLO" (OnReceive) y limpieza opcional de suscriptores inactivos (RemoveInactiveSubscribers_NoLock).

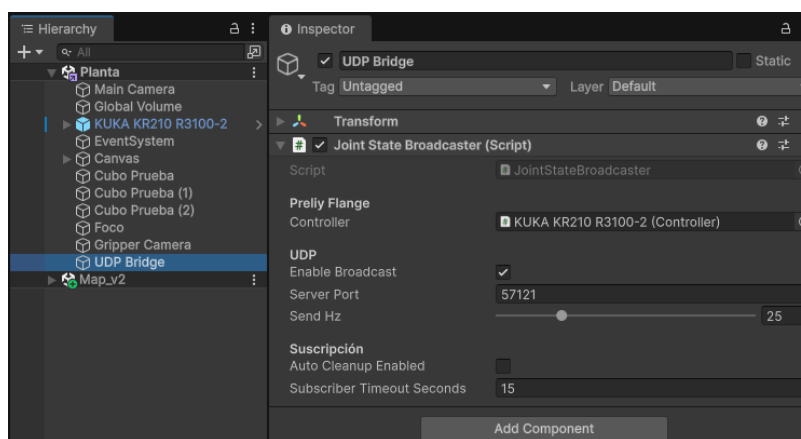


Figura 39: GameObject UDP Bridge: componente JointStateBroadcaster con el Controller de Preliminary Flange, puerto, frecuencia de envío y parámetros de suscripción.

El servidor transmite el estado articular a 25 Hz como un mensaje ASCII por paquete con el formato "q1,q2,q3,q4,q5,q6", donde los seis valores corresponden a los ángulos articulares en grados. Se especifica `CultureInfo.InvariantCulture` en la serialización para garantizar el uso del punto decimal, evitando conflictos con la configuración regional del sistema operativo (Figura 40).


```
public class JointStateBroadcaster : MonoBehaviour
{
    private void SendJointState()
    {
        // Mismo array que JointStatePublisher: JointState.Value[0..5] en GRADOS.
        var jointState = _controller.MechanicalGroup.JointState;

        _messageBuilder.Clear();
        _messageBuilder.Append(jointState.Value[0].ToString("F2", CultureInfo.InvariantCulture));
        _messageBuilder.Append(',');
        _messageBuilder.Append(jointState.Value[1].ToString("F2", CultureInfo.InvariantCulture));
        _messageBuilder.Append(',');
        _messageBuilder.Append(jointState.Value[2].ToString("F2", CultureInfo.InvariantCulture));
        _messageBuilder.Append(',');
        _messageBuilder.Append(jointState.Value[3].ToString("F2", CultureInfo.InvariantCulture));
        _messageBuilder.Append(',');
        _messageBuilder.Append(jointState.Value[4].ToString("F2", CultureInfo.InvariantCulture));
        _messageBuilder.Append(',');
        _messageBuilder.Append(jointState.Value[5].ToString("F2", CultureInfo.InvariantCulture));
        _messageBuilder.Append('\n');

        byte[] datagram = Encoding.ASCII.GetBytes(_messageBuilder.ToString());

        List<IPEndPoint> targets;
        lock (_subscribersLock)
        {
            if (_autoCleanupEnabled)
                RemoveInactiveSubscribers_NoLock();

            targets = new List<IPEndPoint>(_subscribers.Keys);
        }

        foreach (var subscriber in targets)
        {
            try
            {
                _udpClient.Send(datagram, datagram.Length, subscriber);
            }
            catch (SocketException)
            {
                // Un suscriptor caído (ej. puerto ICMP inalcanzable) no debe frenar el envío a los demás.
            }
        }
    }
}
```

Figura 40: SendJointState(): serialización del estado articular con CultureInfo.InvariantCulture y envío por UDP a cada suscriptor registrado.

Se desarrollaron cuatro clientes de referencia que comparten el mismo protocolo de suscripción y reutilizan la misma tabla de parámetros DH y la misma fórmula de calibración mecánica (JointConfig.Offset/Factor).

El primero, `udp_joint_receiver.py`, es el cliente de consola más simple: solicita por consola la IP y puerto del servidor, envía el datagrama de suscripción inicial y mantiene un hilo auxiliar de *keep-alive* que repite el "HELLO" cada 5 s. Al recibir cada paquete valida que contenga exactamente 6 campos numéricos e imprime los ángulos con marca de tiempo.

El segundo, `udp_robot_viewer.py`, añade visualización 3D desde Python utilizando el *Robotics Toolbox for Python* de Peter Corke (`roboticstoolbox-python`) con el backend `pyplot` (`matplotlib`). El backend *Swift* (`three.js`) fue descartado porque requiere geometría STL/URDF; el modelo puramente cinemático (DHRobot) construido a partir de los parámetros DH solo es graficable con `pyplot`, que dibuja el robot como figura de palillos (*stick figure*) a partir de la cinemática directa, igual que MATLAB. El script drena el buffer de recepción en cada ciclo para procesar únicamente el paquete

más reciente y evitar el lag acumulado, y superpone en la figura un panel de texto con los seis ángulos articulares actuales en grados, tal como llegan de Unity. La Figura 41 muestra el resultado.

El tercero, `robot_udp_plot.m`, reproduce el mismo mecanismo de suscripción en MATLAB y anima el modelo cinemático del KR210 con la librería *Robotics Toolbox* de Peter Corke (**SerialLink**), aplicando idéntica estrategia de drenado de buffer. La Figura 42 muestra la réplica resultante.

El cuarto es un **dashboard web** implementado en Python con servidor HTTP embebido (`udp_robot_dashboard.py`), accesible desde cualquier navegador en la misma red. La interfaz muestra el estado articular como gauges circulares para cada articulación (J1–J6), una vista esquemática 3D interactiva del robot y un indicador de paquetes recibidos. El mismo dashboard se ejecuta localmente en la PC (Figura 43) o se sirve a dispositivos móviles en la red Tailscale (Figura 44).

La comunicación se verifica sobre la red virtual **Tailscale**, que conecta mediante VPN el equipo con Unity, el PC de MATLAB/Python y el teléfono, permitiendo operar los cuatro clientes simultáneamente con el servidor en ejecución.

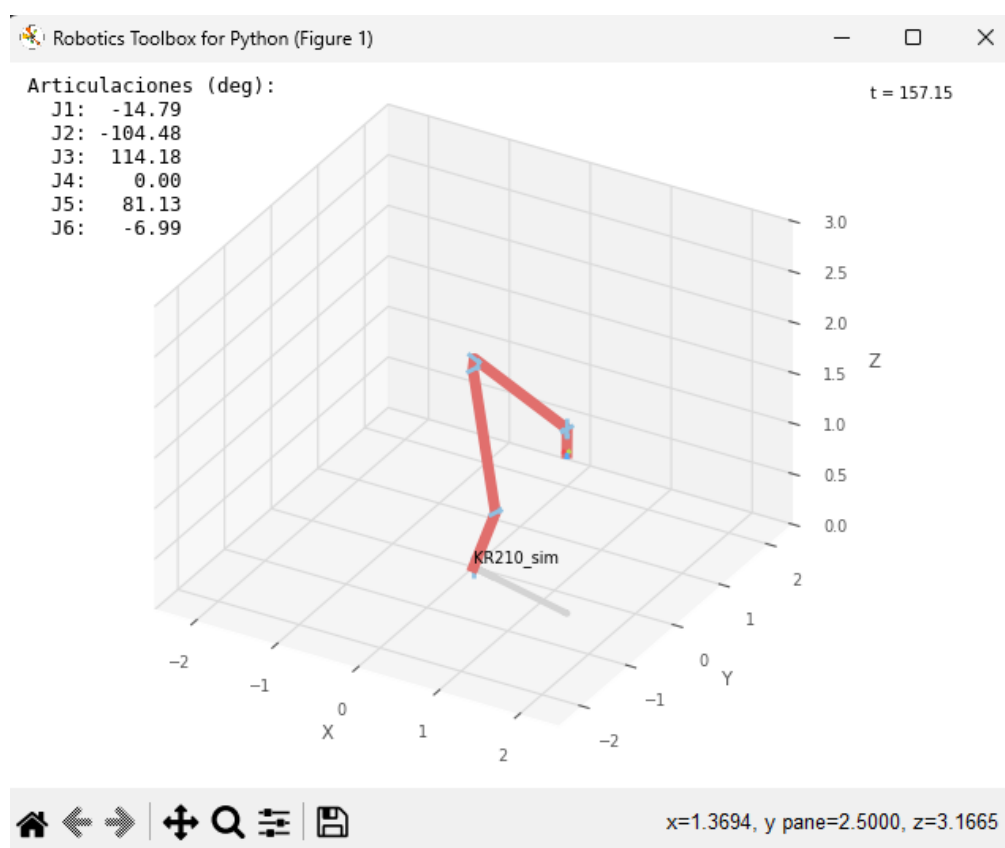


Figura 41: Visor 3D en Python: el script `udp_robot_viewer.py` anima el modelo cinemático del KR210 en tiempo real usando *Robotics Toolbox for Python* (backend `pyplot`) y muestra los ángulos articulares actuales recibidos vía UDP desde Unity.

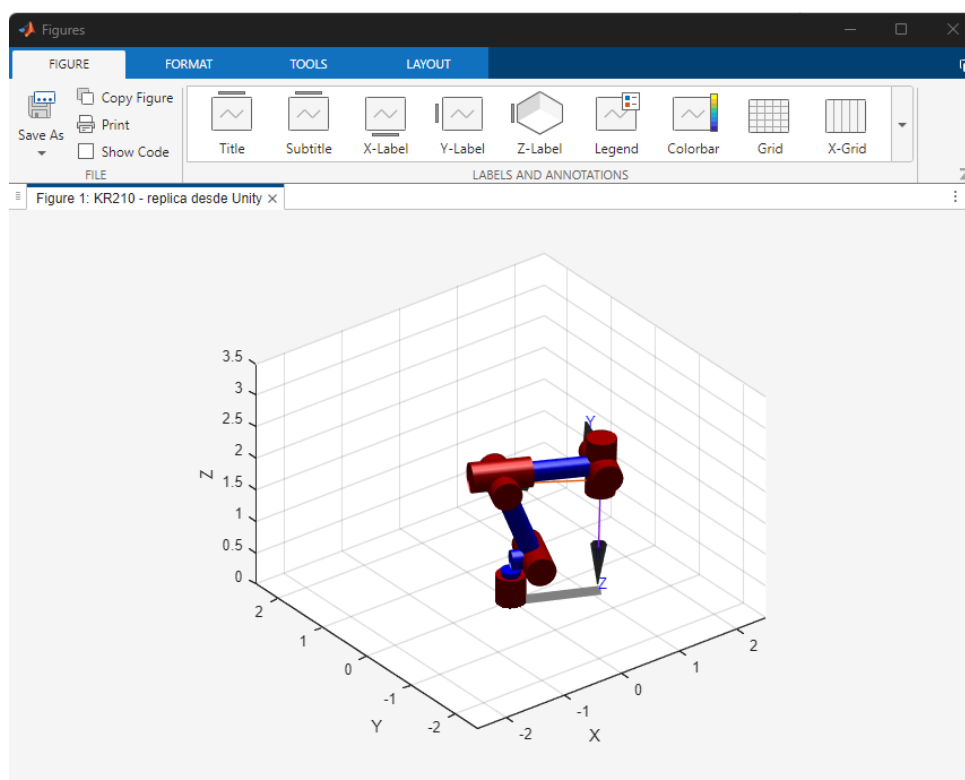


Figura 42: Réplica del estado articular del KR210 en MATLAB mediante el *Robotics Toolbox* de Peter Corke, recibida vía UDP desde Unity en tiempo real.

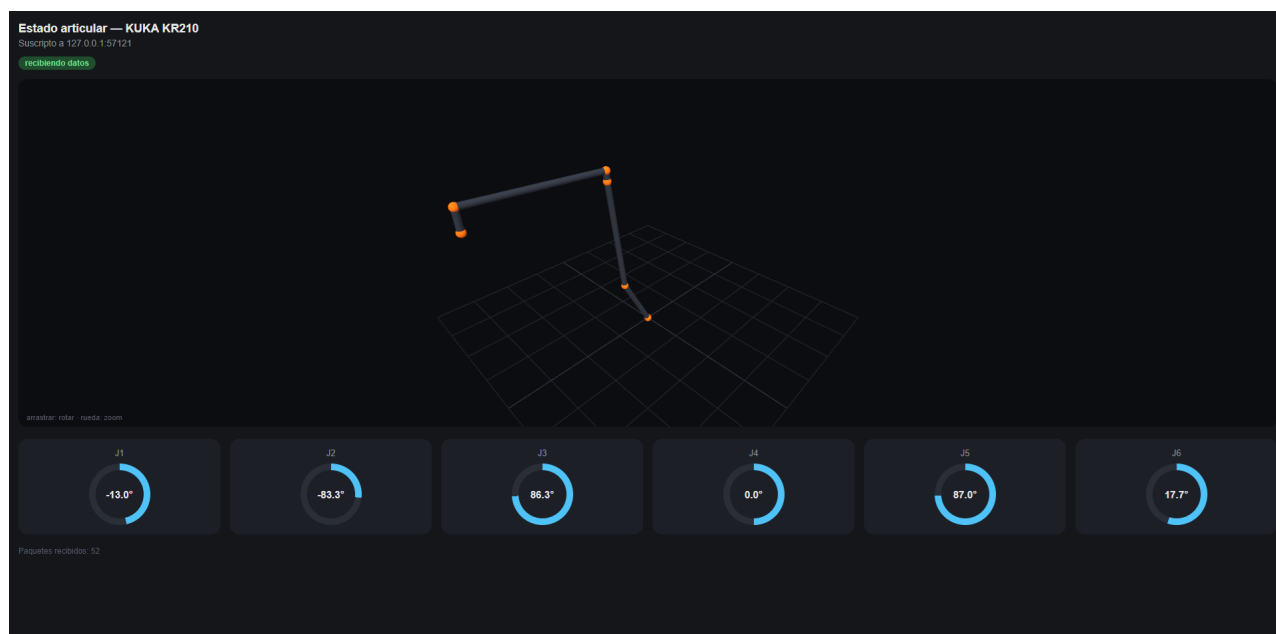


Figura 43: Dashboard web ejecutado en PC: muestra los ángulos articulares J1–J6 como gauges circulares, una vista esquemática 3D interactiva y el conteo de paquetes recibidos. Accesible desde cualquier navegador en la red Tailscale.

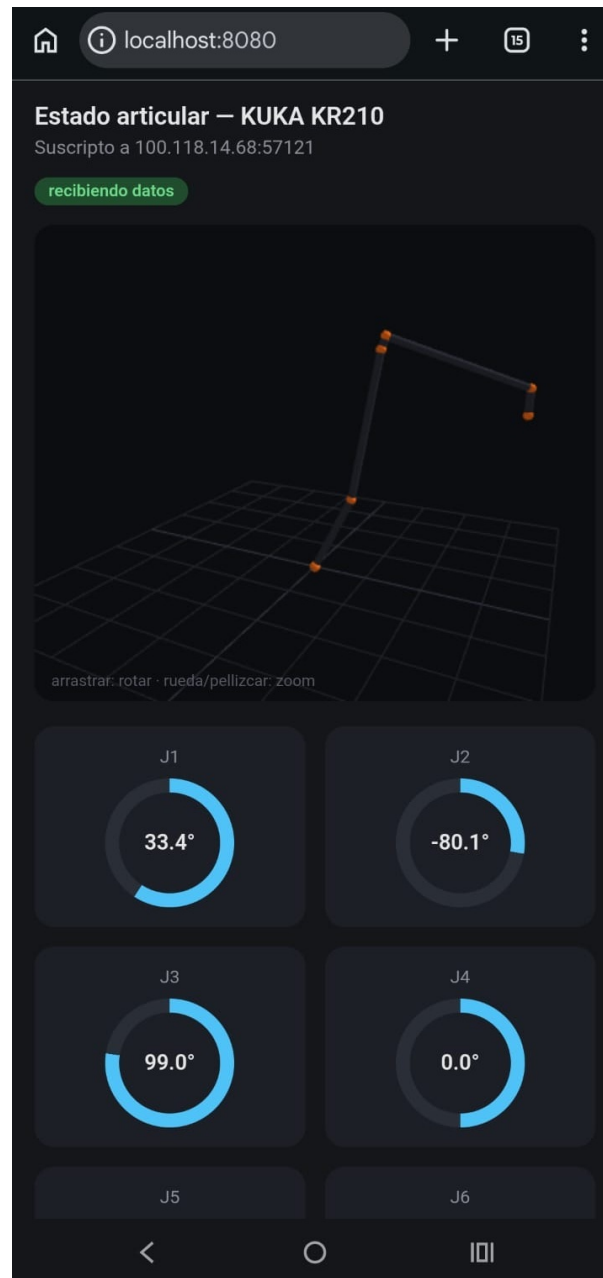


Figura 44: Dashboard web móvil: misma aplicación servida al teléfono vía Tailscale, con la vista 3D y los gauges articulares en formato vertical.